



Spring Data

1

Spring Data

2

- ✓ Présentation
- ✓ Spring Data et Maven
- ✓ Spring Data Jdbc
- ✓ Spring Data JPA
- ✓ Projection
- ✓ Version et Lock
- ✓ Criteria / Example
- ✓ Spring Data REST
- ✓ Spring Data MongoDB

Présentation

- Spring Data a pour objectif de vous simplifier l'accès à vos données.
- Elles peuvent provenir de :
 - JDBC
 - JPA
 - LDAP
 - MongoDB, Cassandra, Redis, Elasticsearch, Neo4J
 - Services REST
 - VMWare GemFire
 - Couchbase
 - Envers

Présentation

- Spring Data va **simplifier** le codage des DAOs
- Soit par :
 - Héritage d'une classe qui fait déjà du CRUD
 - Utilisation d'annotations qui vont simplifier le code
 - Utilisation d'interfaces qui seront introspectées par le Spring afin d'en générer le code des DAOs dynamiquement
- Objectif : zéro ligne de code dans ses DAOs
 - Possible uniquement dans les cas simples
- Plein d'exemples disponibles :
 - <https://github.com/spring-projects/spring-data-examples>

Spring Data et Maven (sans SpringBoot)

➤ Dépendances Maven simplifiées :

```
<dependencyManagement>
  <dependencies>
    <!-- Afin d'éviter d'indiquer tous les éléments du Spring -->
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-framework-bom</artifactId>
      <version>${version.spring}</version>
      <scope>import</scope>
      <type>pom</type>
    </dependency>
    <!-- Afin d'éviter les problématiques de dépendances Spring Data -->
    <!-- Ne marche pas avec Spring JDBC -->
    <dependency>
      <groupId>org.springframework.data</groupId>
      <artifactId>spring-data-releasetrain</artifactId>
      <version>${version.spring-data-releasetrain}</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>
```

Le <dependencyManagement> ne vous épargnera pas de placer une dépendance dans <dependencies>

La version sera dépendante de votre version de Spring

Spring Data et Maven (sans SpringBoot)

- En fonction de ce que vous voulez faire, vous ajouterez dans `<dependencies>` vos `groupId` et `artifactId` sans préciser les versions
- Exemple : Spring Data JPA

```
<dependencies>
  <dependency>
    <groupId>org.springframework.data</groupId>
    <artifactId>spring-data-jpa</artifactId>
    <!-- Version gérée par le dependencyManagement -->
  </dependency>
```

D'une version à l'autre

- Ne changez pas votre version de Spring Data de manière désinvolte
 - Pour rappel la version est dépendante de Spring
- Il y a de gros changements d'une version majeure à l'autre
 - Méthodes en plus
 - Méthodes en moins
- Rien d'insurmontable, mais de quoi vous obliger à modifier votre code.

Spring Data : JDBC

➤ Repose sur le module/framework

➤ `org.springframework.data / spring-data-jdbc`

➤ Avantages :

- Vous avez un DAO CRUD déjà codé via l'interface `org.springframework.data.repository.CrudRepository<T, ID>`
 - ❑ T représente la classe de l'entité
 - ❑ ID représente la classe de la clef primaire associée à l'entité T
- Vos requêtes sont en SQL directement dans une annotation **`@org.springframework.data.jdbc.repository.query.Query`**
- Vos entités peuvent être automatiquement mappées
 - ❑ Donc pas de RowMapper à faire (sauf cas complexes)
 - ❑ Ne prend pas en compte tous les types de jointures

Spring Data : JDBC

➤ Exemple d'entité: Il n'y a **pas besoin** de RowMapper

```
7 package com.banque.entity.impl;
8
9 import org.springframework.data.relational.core.mapping.Column;
10 import org.springframework.data.relational.core.mapping.Table;
11
12 /**
13  * Le bean qui represente un Compte. <br>
14  */
15 @Table("compte")
16 public class CompteEntity extends AbstractEntity {
17
18     private static final long serialVersionUID = 1L;
19
20     @Column("libelle")
21     private String libelle;
22     @Column("solde")
23     private Double solde;
24     @Column("decouvert")
25     private Double decouvert;
26     @Column("taux")
27     private Double taux;
28     @Column("utilisateurId")
29     private Integer utilisateurId;
```

Attention : aux
majuscules / minuscules

Spring Data : JDBC

- Exemple de DAO : Il n'y a aucune classe qui implémente cette interface

```
7 package com.banque.dao;
8
9 import java.util.List;
10 import java.util.Optional;
11
12 import org.springframework.data.jdbc.repository.query.Query;
13 import org.springframework.data.repository.CrudRepository;
14 import org.springframework.data.repository.query.Param;
15 import org.springframework.stereotype.Repository;
16
17 import com.banque.entity.impl.CompteEntity;
18
19 /**
20  * Compte DAO en utilisant Spring Data avec JPA. <br>
21  * Pas besoin de code, juste les requetes en JPA.
22  */
23 @Repository
24 public interface ICompteDAO extends CrudRepository<CompteEntity, Integer> {
25     /**
26      * Selectionne tous les comptes qui appartiennent a un utilisateur.
27      *
28      * @param aUserId
29      *      un utilisateur id
30      * @return tous les comptes de l'utilisateur
31      */
32     @Query("select * FROM Compte cpt where cpt.utilisateurId = :aUserId order by cpt.libelle asc")
33     public Optional<List<CompteEntity>> findByUtilisateurIdOrderByLibelleAsc(@Param("aUserId") int aUserId);
34 }
```

Attention : sous postgres par exemple, il faut écrire `cpt."utilisateurId"`, afin de prendre en compte le *case sensitive*

Ma requête en SQL

Spring Data : JDBC

Classe de Configuration

- Rien de particulier à préciser hormis le nom du/des packages contenant vos @Repository (via **@EnableJdbcRepositories**)

```
@Configuration
@PropertySource("classpath:spring/database.properties")
@ComponentScan({ "com.banque.service" })
@EnableJdbcRepositories({ "com.banque.dao" })
@EnableTransactionManagement
public class SpringConfiguration {

    // Data Source et Transaction Manager

}
```

Spring Data : JPA

➤ Dépendance sur

➡ `org.springframework.data / spring-data-jpa`

➤ Avantages :

- ➡ Vous avez un DAO CRUD déjà codé via l'interface `org.springframework.data.repository.CrudRepository<T, ID>`
 - ❑ Il a même des enfants comme **PagingAndSortingRepository** ou **JpaRepository**
- ➡ Vos requêtes seront auto-générées
 - ❑ Via le JPQL de l'annotation **@org.springframework.data.jpa.repository.Query**
 - ❑ Via le nom de votre méthode qui devra respecter les contraintes Spring Data
- ➡ Vous pouvez appeler des procédures stockées via l'annotation **@org.springframework.data.jpa.repository.query.Procedure**

Spring Data : JPA

- Exemple d'entité: Rien ne change (comparativement à du JPA classique)

```
7 package com.banque.entity.impl;
8
9 import java.math.BigDecimal;
10 import java.util.Set;
11
12 import javax.persistence.*;
13
14 /**
15  * Le bean qui represente un Compte. <br>
16  */
17 @Entity
18 @Table(name = "compte")
19 public class CompteEntity extends AbstractEntity {
20
21     private static final long serialVersionUID = 1L;
22
23     @Column(name = "libelle", length = 250)
24     private String libelle;
25     @Column(name = "solde", precision = 10)
26     private BigDecimal solde;
27     @Column(name = "decouvert", precision = 10)
28     private BigDecimal decouvert;
29     @Column(name = "taux", precision = 5)
30     private BigDecimal taux;
31
32     @OneToMany(mappedBy = "compte", fetch = FetchType.LAZY, cascade = CascadeType.ALL, orphanRemoval = true)
33     private Set<OperationEntity> operations;
34     @ManyToOne
35     @JoinColumn(name = "utilisateurId")
36     private UtilisateurEntity utilisateur;
```

Spring 5 = javax.*
Spring Boot < 3

Spring 6 = jakarta.*
Spring Boot >= 3

Spring Data : JPA

➤ Exemple de DAO

➡ Il n'y a aucune classe qui implémente cette interface

```
1 package com.banque.dao;
2
3 import java.util.List;
4
5 import org.springframework.data.jpa.repository.Query;
6 import org.springframework.data.repository.PagingAndSortingRepository;
7 import org.springframework.data.repository.query.Param;
8 import org.springframework.stereotype.Repository;
9
10 import com.banque.entity.impl.CompteEntity;
11
12 * Compte DAO en utilisant Spring Data avec JPA. <br/>
13 @Repository
14 public interface ICompteDAO extends PagingAndSortingRepository<CompteEntity, Integer> {
15     * Selectionne tous les comptes qui appartiennent a un utilisateur.
16     @Query("FROM CompteEntity cpt where cpt.utilisateur.id = :aUserId order by cpt.libelle")
17     public List<CompteEntity> findAllBelongToUserId(@Param("aUserId") int aUserId);
18 }
```

Ma requête en JPQL

Spring Data : JPA

➤ Exemple de DAO

- ➡ Il n'y a aucune classe qui implémente cette interface
- ➡ Ici, pas de requête du tout !

```
1 package com.banque.dao;
2
3 import java.util.List;
4
5 import org.springframework.data.repository.CrudRepository;
6 import org.springframework.stereotype.Repository;
7
8 import com.banque.entity.impl.CompteEntity;
9
10 @Repository
11 public interface ICompteDAO extends CrudRepository<CompteEntity, Integer> {
12
13     /**
14      * Le Spring Data va fabriquer pour vous la requête en regardant le nom
15      * méthode :<br/>
16      * find : select<br/>
17      * By : from<br/>
18      * Customer : nom de l'objet visé<br/>
19      */
20     public List<CompteEntity> findByCompteEntity(CompteEntity ce);
21 }
```

Ma requête en
mots clefs
Spring

Spring Data : JPA

16

➤ Mots clefs supportés :

And	<code>findByLastnameAndFirstname</code>	<code>where x.lastname = ?1 and x.firstname = ?2</code>
Or	<code>findByLastnameOrFirstname</code>	<code>where x.lastname = ?1 or x.firstname = ?2</code>
Is,Equals	<code>findByFirstname</code> , <code>findByFirstnames</code> , <code>findByFirstnameEquals</code>	<code>where x.firstname = ?1</code>
Between	<code>findByStartDateBetween</code>	<code>where x.startDate between ?1 and ?2</code>
LessThan	<code>findByAgeLessThan</code>	<code>where x.age < ?1</code>
LessThanEqual	<code>findByAgeLessThanEqual</code>	<code>where x.age <= ?1</code>
GreaterThan	<code>findByAgeGreaterThan</code>	<code>where x.age > ?1</code>
GreaterThanEqual	<code>findByAgeGreaterThanEqual</code>	<code>where x.age >= ?1</code>
After	<code>findByStartDateAfter</code>	<code>where x.startDate > ?1</code>
Before	<code>findByStartDateBefore</code>	<code>where x.startDate < ?1</code>
IsNull	<code>findByAgeIsNull</code>	<code>where x.age is null</code>
IsNotNull,NotNull	<code>findByAge(Is)NotNull</code>	<code>where x.age not null</code>
Like	<code>findByFirstnameLike</code>	<code>where x.firstname like ?1</code>
NotLike	<code>findByFirstnameNotLike</code>	<code>where x.firstname not like ?1</code>
StartingWith	<code>findByFirstnameStartingWith</code>	<code>where x.firstname like ?1 (parameter bound with appended %)</code>
EndingWith	<code>findByFirstnameEndingWith</code>	<code>where x.firstname like ?1 (parameter bound with prepended %)</code>
Containing	<code>findByFirstnameContaining</code>	<code>where x.firstname like ?1 (parameter bound wrapped in %)</code>
OrderBy	<code>findByAgeOrderByLastnameDesc</code>	<code>where x.age = ?1 order by x.lastname desc</code>
Not	<code>findByLastnameNot</code>	<code>where x.lastname <> ?1</code>
In	<code>findByAgeIn(Collection<Age> ages)</code>	<code>where x.age in ?1</code>
NotIn	<code>findByAgeNotIn(Collection<Age> age)</code>	<code>where x.age not in ?1</code>
True	<code>findByActiveTrue()</code>	<code>where x.active = true</code>
False	<code>findByActiveFalse()</code>	<code>where x.active = false</code>
IgnoreCase	<code>findByFirstnameIgnoreCase</code>	<code>where UPPER(x.firstname) = UPPER(?1)</code>

Spring Data : JPA

➤ Bonne pratique :

- Quand vous réalisez une méthode qui doit récupérer **un seul** élément
 - ❑ `Optional<T> findMonElement(...)` ⇔ **Faire** usage de `Optional`
- Quand vous réalisez une méthode qui doit récupérer **plusieurs** éléments
 - ❑ `List<T> findMesElements(...)` ⇔ **NE PAS faire** usage de `Optional`

➤ Inconvénients :

- Vos paramètres de requêtes ne peuvent pas être nuls
 - ❑ Peut entrainer un code lourd côté service métiers

➤ Une grosse documentation

- <https://docs.spring.io/spring-data/jpa/reference/>

Spring Data : JPA Configuration XML



- Rien de particulier à préciser hormis le nom du/des packages contenant vos @Repository (**<jpa:repositories />**)

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:jpa="http://www.springframework.org/schema/data/jpa"
  xsi:schemaLocation="http://www.springframework.org/schema/beans http://www.springframework.org/schema/beans/spring-beans.xsd
    http://www.springframework.org/schema/data/jpa http://www.springframework.org/schema/data/jpa/spring-jpa.xsd">

  <jpa:repositories base-package="com.banque.dao" />

  <bean id="entityManagerFactory" class="org.springframework.orm.jpa.LocalContainerEntityManagerFactoryBean">
    ...
  </bean>
```

Spring Data : JPA

Classe de Configuration



- Rien de particulier à préciser hormis le nom du/des packages contenant vos `@Repository` (via `@EnableJpaRepositories`)

```
@Configuration
@PropertySource("classpath:spring/database.properties")
@ComponentScan({ "com.banque.service" })
@EnableJpaRepositories({ "com.banque.dao" })
@EnableTransactionManagement
public class SpringConfiguration {

    // Data Source + Entity Manager Factory + Transaction Manager
}
```

Travaux pratiques

- Exo 17 - Reprenez les TPs JDBC et JPA
 - Faites-le passer en Spring Data



Spring Data – Version et Lock

- En JPA classique, l'annotation **@Version** indique un attribut qui servira pour le versionning de l'entité en base
- @Version permet de gérer les conflits entre versions d'entités lors des update/delete
- Spring Data ne s'occupe pas de cet élément, cela reste la responsabilité du JPA

Spring Data – Version et Lock

- En JPA pour la gestion des verrous on utilise LockModeType (OPTIMISTIC ou PESSIMISTIC_WRITE) lors des appels aux méthodes de l'entity manager
- En Spring Data JPA vous pouvez faire usage de l'annotation **@Lock** sur vos méthodes de repository

```
@Repository
public interface AccountRepository extends JpaRepository<Account, Long> {

    @Lock(LockModeType.PESSIMISTIC_WRITE)
    @Query("select a from Account a where a.id = :id")
    Optional<Account> findByIdForUpdate(@Param("id") Long id);
}
```

Spring Data – Version et Lock

- **@Lock** pour bien fonctionner aura besoin
 - D'une transaction (au niveau @Service en principe)
 - Selon le type de lock : de @Version (au niveau de votre entité)
- Les erreurs remontées peuvent varier selon les cas
 - LockTimeoutException
 - TransactionRequiredException
 - OptimisticLockException

Spring Data - Projections

- La **projection** est une manière de projeter la vision que Spring Data aura sur une de vos entités
- A travers de simples interfaces (au sens Java), vous pouvez indiquer à Spring Data comment réaliser le mapping
- Avec cette approche, si vous n'avez besoin que du nom et du prénom d'un client, votre requête sera optimisée pour n'aller chercher que ce que vous avez demandé

Spring Data - Projections

- Par exemple, mon entité Compte qui représente ma table Compte :

```
@Entity
@Table(name = "compte")
public class CompteEntity {

    @Column(name = "libelle", length = 250)
    private String libelle;
    @Column(name = "solde", precision = 10)
    private BigDecimal solde;
    @Column(name = "decouvert", precision = 10)
    private BigDecimal decouvert;
    @Column(name = "taux", precision = 5)
    private BigDecimal taux;

    @OneToMany(mappedBy = "compte", fetch = FetchType.LAZY, cascade = CascadeType.ALL, orphanRemoval = true)
    private Set<OperationEntity> operations;
    @ManyToOne
    @JoinColumn(name = "utilisateur_id")
    private UtilisateurEntity utilisateur;

    // ...
}
```

Spring Data - Projections

➤ Une projection sur mon entité Compte

```
public interface CompteSimplifie {  
    String getLibelle();  
    BigDecimal getSolde();  
}
```

➤ Son utilisation dans mon repository Compte

```
@Repository  
public interface ICompteDAO extends JpaRepository<CompteEntity, Integer> {  
  
    List<CompteEntity> findByUtilisateurIdOrderByLibelleAsc(int aUserId);  
  
    List<ICompteSimplifie> findByUtilisateurIdOrderByLibelle(int aUserId);  
}
```

Spring Data - Projections

- Au lieu d'interface, vous pouvez aussi faire usage de **record**
- Vous pouvez aussi utiliser des projections **ouvertes** (dont les informations seront calculées)

```
public interface CompteEnProjectionOuverte {  
    @Value("#{target.libelle + ' ' + target.solde}")  
    String getLibelleDetaille();  
}
```

- Ou dynamique (via la généricité)

```
@Repository  
public interface ICompteDAO extends JpaRepository<CompteEntity, Integer> {  
  
    List<T> findByUtilisateurIdOrderByLibelleAsc(int aUserId, Class<T> type);  
}
```

Spring Data : MongoDB

- Spring Data prend en charge les bases NoSQL comme MongoDB
- Dépendance sur
 - `org.springframework.data / spring-data-mongodb`
- Avantages :
 - Vous pouvez vous passer des classes de DAOs (et du *MongoTemplate*)
 - Vos interfaces de DAOs doivent implémenter `org.springframework.data.mongodb.repository.MongoRepository<E, K>`
 - ❑ E représente la classe/interface des entités
 - ❑ K représente la classe/interface de la clef primaire associée à l'entité
 - Vos requêtes seront auto-générées
 - ❑ Via le nom de votre méthode qui devra respecter les contraintes Spring Data de MongoDB

Spring Data : MongoDB

- Pour vos DAOs vous pouvez
 - Faire usage des noms de méthodes
 - Faire usage de requêtes Mongo en Json via **@org.springframework.data.mongodb.repository.Query** (attention à l'import, on n'est PAS en JPQL)
 - Faire usage de QueryDSL
 - ❑ Utilisation d'objets Predicate
 - ❑ <http://www.querydsl.com/>
- Attention : pas de Cascade All ou Remove orphan
 - Il faudra gérer vous-même ces contraintes

Spring Data : MongoDB

- Rappel : Mapping des entités
 - Gestion de l'incrémentation des clefs primaires à la main
 - Gestion des jointures sans notion de 1..1, n..m, ...

- Sur l'entité : **@Document**
- Sur la clef primaire : **@Id**
- Sur les 'jointures' : **@DBRef**
 - Possibilité de faire du *lazy*
 - *Il n'est pas conseillé d'en faire usage*

- Attention : pas de Cascade All ou Remove orphan
 - Il faudra gérer vous-même ses contraintes

Spring Data : MongoDB

➤ Exemple d'un DAO :

```
@Repository
public interface IUserDao extends MongoRepository<UserEntity, Integer> {

    public Optional<UserEntity> findOneByEmailAndPassword(String pEmail, String pPassword);

    public Optional<UserEntity> findOneByEmail(String pEmail);

    @Query(count = true, value = "{ '$and' : [{ 'status' : 'ENABLED'}, {'isLunchLady': true}] }")
    public int countLunchLady();
}
```

➤ Et de l'entité associée

```
@Document
public class UserEntity {
    @Id
    private ObjectId id;
    private String email;
    private Boolean isLunchLady;
    private String name;
    private String firstname;
    private Sex sex;
    private EntityState status;
    ...
}
```

Spring Data : MongoDB

- Rien de particulier à préciser hormis le nom du/des packages contenant vos `@Repository` (via `@EnableMongoRepositories`)

```
@SpringBootApplication
@EnableTransactionManagement // A partir Spring 5+
public class SpringBootConfiguration {

    @Bean
    // A partir de Spring 5 uniquement
    public MongoTransactionManager txManager(MongoDbFactory dbFactory) {
        return new MongoTransactionManager(dbFactory);
    }
}
```


Spring Data : MongoDB

➤ Rappel des points contraignants en MongoDB

- Il n'y a pas de clef auto-incrémentée, il faut faire une séquence et la gérer dans son code Java
 - ❑ Les id en MongoDB 5 sont des ObjectId
- Il n'y a pas non plus de transactionnel au sens classique du terme
 - ❑ Pas la peine de placer un @Transactional dans votre code de service
 - ❑ Cependant, la version 5 du Spring permet l'usage du transactionnel dans MongoDB (ne couvre pas 100% des options)

Spring Data : MongoDB

➤ Rappel des points contraignants en MongoDB

- Vous ne pouvez pas faire de requête par mot-clef sur les éléments jointurés (@DBRef)

```
public List<CompteEntity> findByUtilisateurEmailOrderByLibelleAsc(String pEmail);
```

- Ne peut pas fonctionner même s'il y a bien un attribut Email dans Utilisateur (lui-même attribut de CompteEntity)
 - ❑ Associations can only be pointed to directly or via their id property!
- Ne fonctionne pas non plus en @Query

Travaux pratiques

- Exo 17 - La banque en Mongo DB



Spring Data : Focus sur le Example

- L'objet `org.springframework.data.domain.Example`

```
public interface Example<T>
```

- Répond au besoin :

- Si vous avez le besoin de faire des recherches sur différents critères avec ou sans option de tri / paging

```
<S extends T> Optional<S> findOne(Example<S> var1);  
<S extends T> Iterable<S> findAll(Example<S> var1);  
<S extends T> Iterable<S> findAll(Example<S> var1, Sort var2);  
<S extends T> Page<S> findAll(Example<S> var1, Pageable var2);  
<S extends T> long count(Example<S> var1);  
<S extends T> boolean exists(Example<S> var1);
```

Spring Data : Focus sur le Example

- Par exemple, vous voulez trier les utilisateurs qui ont le nom qui commence par "Al"

```
UtilisateurEntity entity=new UtilisateurEntity();  
entity.setNom(debutDuNom);  
Example<UtilisateurEntity> example =  
    Example.of(entity, ExampleMatcher.  
                matchingAll().  
                withMatcher("nom",  
                            ExampleMatcher.  
                                GenericPropertyMatcher.of(ExampleMatcher.StringMatcher.STARTING)));  
monSpringDataRepo.findAll(example, Sort.by(Sort.Order.asc("nom")));
```

Méthode parent du Spring Data
Repository

Spring Data : Focus sur le Example

➤ Restrictions de la classe *Example*

- Il n'est pas possible de regrouper les arguments
- Pour les String, les matchers disponibles sont
 - ☐ matching only
 - ☐ includes exact,
 - ☐ case-sensitive, case-insensitive,
 - ☐ starts, ends,
 - ☐ contains,
 - ☐ regex
- Pour les 'non-string', le seul matcher disponible sera
 - ☐ exact-match

Spring Data : Focus sur le Specification

- Dans le même esprit que **Example**, en JPA on trouve **Specification**.
- Cette approche est issue de l'API criteria du JPA
- Il faudra dans un premier temps ajouter l'implémentation `org.springframework.data.jpa.repository.JpaSpecificationExecutor` à votre interface `@Repository`

```
@Repository  
public interface IUserRepository extends JpaRepository<User, Integer>, JpaSpecificationExecutor<User> {  
}
```

Spring Data : Focus sur le Specification

- Cette interface va ajouter les méthodes suivantes à votre repository :
- long **count**(Specification<T> spec)
 - long **delete**(Specification<T> spec)
 - boolean **exists**(Specification<T> spec)
 - List<T> **findAll**(Specification<T> spec)
 - Page<T> **findAll**(Specification<T> spec, Pageable pageable)
 - List<T> **findAll**(Specification<T> spec, Sort sort)
 - <S extends T,R> R **findBy**(Specification<T> spec, Function<FluentQuery.FetchableFluentQuery<S>,R> queryFunction)
 - Optional<T> **findOne**(Specification<T> spec)

Spring Data : Focus sur le Specification

- L'objet ***Specification*** va vous permettre de définir un ou plusieurs critères
- **ATTENTION** : potentiellement il faudra faire usage des méta-modèles Hibernate pour que cela fonctionne
 - Ajouter une dépendance dans Maven
 - https://docs.hibernate.org/stable/jpamodelgen/reference/en-US/html_single/

Spring Data : Focus sur le Specification

➤ Exemple

```
public class UserSpecs {  
    public static Specification<User> getUserByEmailSpec(String email) {  
        return (root, query, criteriaBuilder) -> {  
            // return criteriaBuilder.equal(root.get(User_.email), email);  
            return criteriaBuilder.equal(root.get("email"), email);  
        };  
    }  
    public static Specification<User> getUserByStatusSpec(EnumStatus status) {  
        return (root, query, criteriaBuilder) -> {  
            return criteriaBuilder.equal(root.get("status"), status);  
        };  
    }  
}
```

Spring Data : Focus sur le Specification

- User_ : méta modèle de la classe entité User
 - Elle est en général générée via l'ajout de la dépendance `jpaamodelgen`
 - ❑ <https://sites.google.com/site/pawneecity/sprint-boot/data-jpa-and-jpaspecificationexecutor-spring-boot>
 - Elle n'est pas obligatoire, on peut passer par le nom des **attributs** dans l'entité (PAS le nom des colonnes)

Spring Data : Focus sur le Specification

➤ Utilisation :

```
@Service
public class UserService {
    @Autowired private UserRepository repository;

    public List<User> rechercherUsers(String email, EnumStatus status) {
        Specification<User> spec = Specification.where(UserSpecs.getUserByStatusSpec(status));

        if (email != null && !email.isEmpty()) {
            spec = spec.and(UserSpecs.getUserByEmailSpec(email));
        }

        return repository.findAll(spec);
    }
}
```

Spring Data : REST

- Dépendance sur
 - `org.springframework.data / spring-data-jpa`
 - `org.springframework.data / spring-data-rest-webmvc`
 - Un framework JSON (Jackson)
- Dans cette approche, le Spring va vous faire un DAO qui sera en plus un Web Service REST
- Le DAO étant en Spring Data, il supportera les opérations en CRUD + vos requêtes
- Les web services seront automatiquement en **HATEOAS**

Spring Data : REST

➤ **HATEOAS :** **Hypermedia As The Engine Of Application State**

- De cette manière votre client REST n'a pas besoin de connaître les web services
 - Il peut les découvrir dynamiquement
 - Le serveur envoie toujours les informations dans un format JSON qui contient lui-même des URLs d'appels
- Le serveur peut changer ses web services sans impacter le client
 - A partir du moment où le client fait l'effort d'utiliser les liens (links/href)

Spring Data : REST Par fichier XML



- Sur un projet Spring Data JPA + Spring MVC classique (avec son DispatcherServlet) il suffit simplement d'ajouter le bean
`org.springframework.data.rest.webmvc.config.RepositoryRestMvcConfiguration`

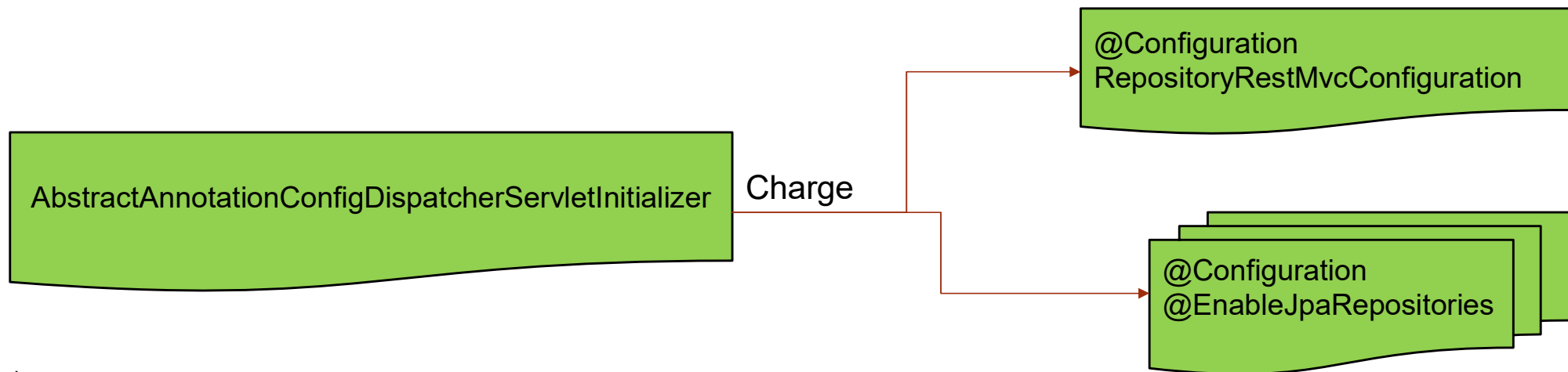
```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <beans xmlns="http://www.springframework.org/schema/beans"
3     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:context="http://www.springframework.org/schema/context"
4     xsi:schemaLocation="http://www.springframework.org/schema/beans
5         http://www.springframework.org/schema/beans/spring-beans.xsd
6         http://www.springframework.org/schema/context
7         http://www.springframework.org/schema/context/spring-context.xsd">
8
9
10    <!-- On declare simplement le bean -->
11    <bean class="org.springframework.data.rest.webmvc.config.RepositoryRestMvcConfiguration" />
12
13 </beans>
```

Spring Data : REST

Configuration par classe



- La classe du Dispatcher se chargera de monter les configurations
 - La configuration RepositoryRestMvcConfiguration devra porter la configuration spécifique aux web services
 - ❑ Et le mapping



Spring Data : REST En Spring Boot

- Il n'y a pas grand-chose à faire hormis ajouter les dépendances nécessaires

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<b><dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-rest</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-hateoas</artifactId>
</dependency>
```

Spring Data : REST

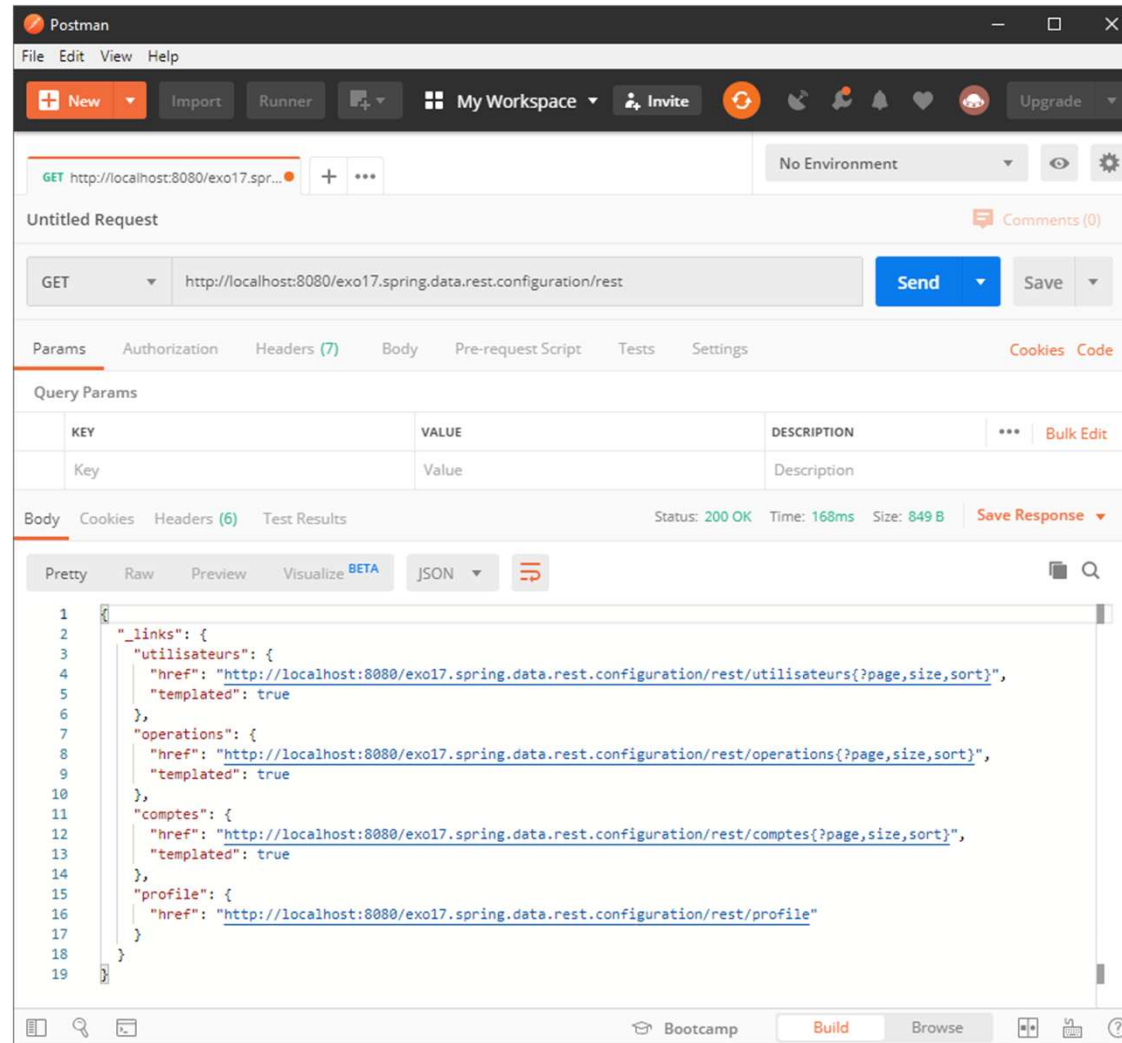
- Sur ces DAO, l'annotation `@Repository` se transforme en `@org.springframework.data.rest.core.annotation.RepositoryRestResource`
- Le reste du DAO en Spring Data ne change pas :

```
1 package com.banque.dao;
2
3 import java.util.List;
4
5 import org.springframework.data.jpa.repository.Query;
6 import org.springframework.data.repository.PagingAndSortingRepository;
7 import org.springframework.data.repository.query.Param;
8 import org.springframework.data.rest.core.annotation.RepositoryRestResource;
9
10 import com.banque.entity.impl.CompteEntity;
11
12 * Compte DAO en utilisant Spring Data avec JPA. <br/>
13 @RepositoryRestResource(collectionResourceRel = "comptes", path = "comptes")
14 public interface ICompteDAO extends PagingAndSortingRepository<CompteEntity, Integer> {
15
16     * Selectionne tous les comptes qui appartiennent a un utilisateur.
17     @Query("FROM CompteEntity cpt where cpt.utilisateur.id = :aUserId order by cpt.libelle")
18     public List<CompteEntity> findAllBelongToUserId(@Param("aUserId") int aUserId);
19 }
```

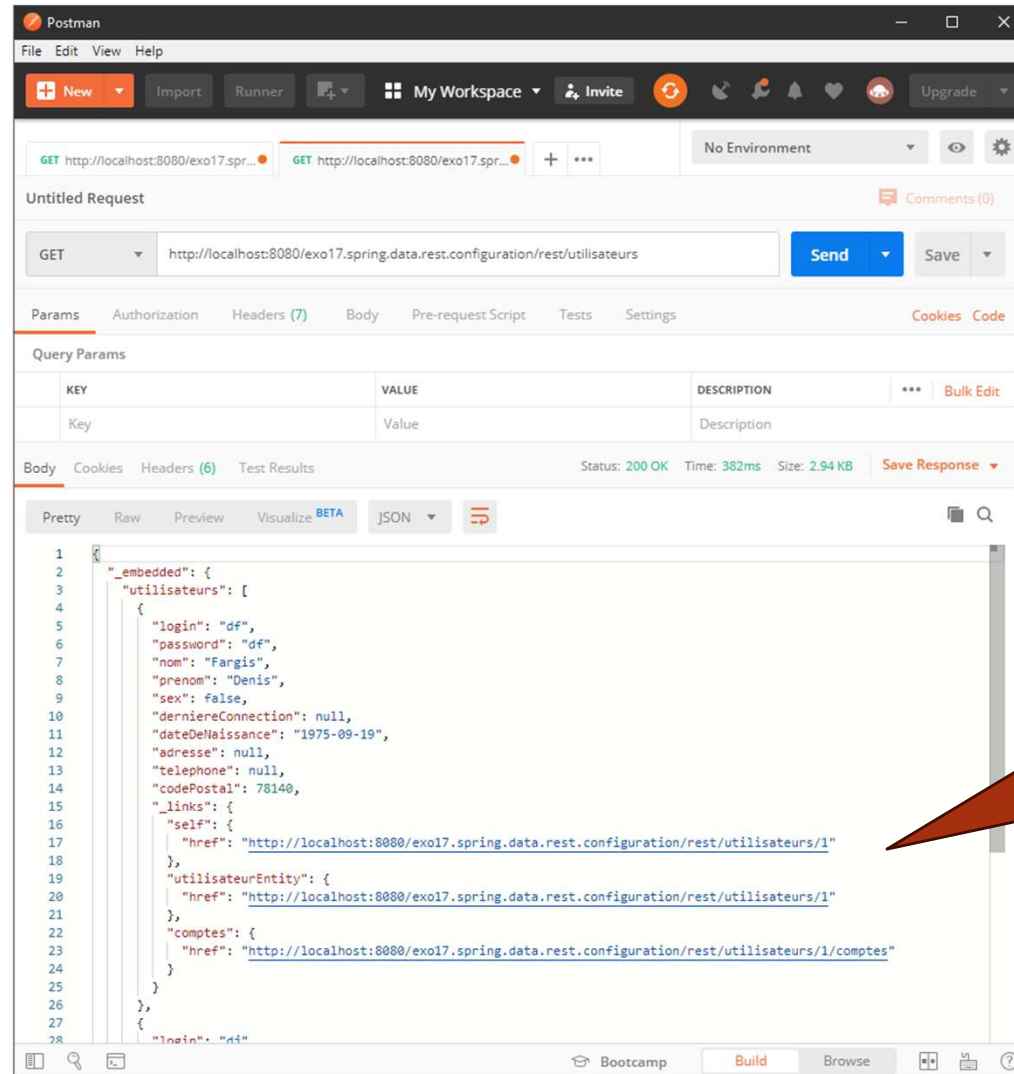
Spring Data : REST

- Lors du déploiement, tous les DAO seront disponibles en tant que contrôleur REST
- Lancer *Postman* pour voir la liste des services disponibles ainsi que les éléments attendus
 - Si vous êtes en Linux ou powershell vous pouvez aussi utiliser *curl* ou *Invoke-RequestMethod*
- Remarque : par défaut, les ID de vos entités ne seront pas exposés dans le flux JSON
 - Ce comportement est paramétrable via la méthode `exposeIdsFor` de `RepositoryRestConfiguration`

Spring Data : REST



Spring Data : REST



Par défaut,
nous sommes
en HAL
(HATEOAS)

Spring Data : REST

- Il n'y a plus vraiment de couche service ni contrôleur
 - Tout est fusionné
- Dans cette approche chaque méthode de vos DAO est **transactionnelle**
 - Gérée pour vous en automatique (pas besoin de @Transactional)
 - Mais vous pouvez garder/déclarer le transactionManager
 - ❑ Il vous servira sûrement pour les tests unitaires afin de forcer le rollback
- Si vous avez besoin de faire des opérations multi-table, la solution la plus simple sera la procédure stockée
 - Votre transaction sera gérée par la base de données

Spring Data : REST

- Par défaut, toutes les méthodes sont exposées comme des end-point
- Pour **cacher** une méthode, faites usage de l'annotation **@RestResource(exported=false)**

```
@RepositoryRestResource(collectionResourceRel = "users", path = "users")
public interface UserRepository extends ListCrudRepository<User, Long> {

    @Override
    @RestResource(exported = false)
    void deleteById(Long id);

    @Override
    @RestResource(exported = false)
    void delete(User entity);
}
```

Spring Data : REST

- Si vous voulez faire le contraire ⇔ cacher toutes les méthodes sauf celles que vous autorisez
 - Dans votre fichier de paramétrage indiquez :
 - ❑ `spring.data.rest.detection-strategy=annotated`
 - Et dans vos méthodes de repository **@RestResource(exported=true)**
- Vous pouvez faire usage de **spring.data.rest.detection-strategy=visibility** pour ne voir que les méthodes publiques
 - Pour rappel, depuis Java 9, il est possible de faire des méthodes private default

Spring Data : REST

- Toujours via **@RestResource**, vous pouvez aussi modifier le nom de la route

```
@RepositoryRestResource(path = "users")
public interface UserRepository extends JpaRepository<User, Long> {

    // Default would be: GET /users/search/findByEmail
    // Changed to: GET /users/search/by-email
    @RestResource(path = "by-email", rel = "by-email")
    User findByEmail(@Param("email") String email);
}
```

Spring Data : REST

- Si vous souhaitez faire des routes spécifiques (commande HTTP / URL dynamiques ...) il est préférable de faire une **classe** annotée en `@RepositoryRestResource`

```
@RepositoryRestController
@RequestMapping("/users")
public class CustomUserController {

    private final UserRepository repository;

    public CustomUserController(UserRepository repository) {
        this.repository = repository;
    }

    @PostMapping("/{id}/deactivate")
    public @ResponseBody ResponseEntity<?> deactivateUser(@PathVariable Long id) { ... }
}
```

Spring Data : REST

- Avantages de la classe en `@RepositoryRestResource`
 - Vous pouvez faire usage `PersistentEntityResourceAssembler` directement dans vos méthodes (Spring l'injectera en tant que paramètre)
 - ❑ Son paramétrage est automatique (pas de `linkTo` / `methodOn` à faire)
 - ❑ L'objectif est de rester HATEOAS dans l'ensemble de vos flux

```
@RepositoryRestController
@RequestMapping("/users")
public class CustomUserController {

    @PutMapping("/{id}")
    public @ResponseBody ResponseEntity<?> customUpdate(
        @PathVariable Long id,
        PersistentEntityResourceAssembler assembler) {

        User updatedUser = ...;

        return ResponseEntity.ok(assembler.toModel(updatedUser));
    }
}
```

Travaux pratiques



- Exo 17 - Reprenez le TP sur Spring MVC Rest
 - Remplacez vos DAO par ceux de l'exercice Spring Data JPA
 - Supprimez vos webs services Rest
 - Faites-le passer en Spring Data Rest vos DAOs, testez avec Postman
- Remarque : pour la transformation des dates, vous pouvez faire usage de l'annotation `@org.springframework.format.annotation.DateTimeFormat` sur vos paramètres de DAO
 - Vous pouvez aussi faire usage de la mécanique de votre API Rest
 - ❑ Par exemple `@JsonSerialize` si vous êtes en Jackson